

AUTOMATION IN SOFTWARE DEVELOPMENT · V SEM · UNIT 2 ·
ASSIGNMENT A1

AI Coding Assistant Practicum

Prompt Engineering in Practice — with a tool other than GitHub Copilot

Student	Ojas Shelke — Roll No. 24BTRAO051
AI tool used	Hermes Agent (Nous Research) — agentic CLI coding assistant
Task option	C — small REST API (FastAPI) with CRUD, input validation, unit tests
Project	TabSplit — expense-splitting API for groups (who-owes-whom settlement)
Repository	github.com/Trapston3/AIDD-Unit2-A1-24BTRAO051-OjasShelke (public)
Evidence	10 incremental commits · 13/13 tests passing · 12 screenshots · full verbatim prompt log

Submitted as one PDF compiled from the repository's docs/ folder · Generated 2026-08-26

1 · The tool: Hermes Agent (CLI)

Every artifact in this project was produced by prompting **Hermes Agent**, an agentic CLI coding assistant (invoked as `hermes --yolo -z "<prompt>"`). It plans, writes files, runs the test suite itself, and reports a verification summary. Raw transcripts are preserved verbatim in `docs/responses/`; the exact prompt files in `docs/prompts/`; the graded commit history tags each step with its technique.

2 · Prompt log (summary table)

#	Technique	Exact prompt (verbatim files in docs/prompts/)	Tool response + evaluation
1	Zero-shot	"Create a FastAPI application at <code>src/tabsplit/main.py</code> for a small expense-splitting API called <code>TabSplit</code> ... in-memory dict store; Pydantic models <code>Group/Expense</code> ; <code>POST /groups (201)</code> , <code>GET /groups/{id} (404)</code> , <code>POST members</code> , <code>POST expenses (amount > 0 else 422)</code> , <code>GET /health</code> ..."	Complete working app in one shot; self-verified 12/12 behavior checks. Accepted as-is . Standard boilerplate → zero-shot ideal.
2	Zero-shot	"Create <code>tests/test_main.py</code> and <code>tests/conftest.py</code> ... <code>TestClient</code> ... health ok; <code>201 + int id</code> ; <code>404 missing</code> ; member appends; <code>0/negative → 422</code> ; missing group → <code>404</code> . Independent tests via fixture."	7 tests + store-reset fixture; all green first run. Accepted as-is .
3	Few-shot	4 exact input/output JSON examples defining <code>{"error": {code, message}}</code> with codes <code>GROUP_NOT_FOUND / INVALID_AMOUNT / VALIDATION_ERROR</code> (full text: <code>step03_fewshot_error_envelope.txt</code>)	Handlers for <code>HTTPException + RequestValidationError</code> ; all 4 examples replayed byte-for-byte correct. One legacy test updated by me to the new contract. Accepted with edits .
4	Few-shot	3 style examples (Google docstring template, test docstring, fixture) + rules (<code>≤80-char first line</code> , <code>Raises only when real</code>). Two earlier attempts failed on provider errors.	25 docstrings rewritten; tool audited itself with a transitive raise-graph. Accepted as-is . Conventions = few-shot payoff.
5	Chain-of-thought	"Think step by step before writing any code: restate the problem, work the example arithmetic on paper, decide data structures, THEN implement." + netting contract + worked example	Greedy largest-debtor→creditor netting, <code>≤ D+C-1</code> transfers, dust filter, cents rounding; self-verified 9 properties. Accepted as-is .
6	Chain-of-thought	RED test supplied: settlement of <code>{A: -1.005, B: 1.005}</code> must equal <code>1.01</code> . "Run the suite, capture the failure, explain step by step how Python's <code>round()</code> treats halfway values given binary floating point."	Traced root cause (<code>1.005 → stored as 1.00499...</code> ; round-half-to-even snaps down) and fixed via <code>Decimal(str(v)).quantize(ROUND_HALF_UP)</code> . Accepted as-is . CoT made the reasoning inspectable.
7	Zero-shot	"Create <code>tests/test_netting.py</code> additions: empty dict → <code>[]</code> ; all-zero → <code>[]</code> ; unequal pair → one transfer of smaller amount; three debtors conserve total; amounts always 2 decimals. Follow existing docstring convention."	5 more tests; 13/13 green; convention followed unprompted. Accepted as-is .

Full verbatim prompts and responses: `docs/PROMPT_LOG.md` and `docs/prompts/` · `docs/responses/` in the repository.

3 · Evidence — Zero-shot

● ● ● Hermes Agent CLI - Step 1 (Zero-shot): TabSplit FastAPI scaffold

Ad-hoc verification re-run on the current state of `src/tabsplit/main.py` - passing, but this is a one-off check, not a test suite.

Method: temp script `hermes-verify-npage7l.py` created via `tempfile.mkstemp` in the OS temp dir, executed as a fresh subprocess against the changed module, deleted afterward (cleanup confirmed).

Result: 12/12 passed, exit 0.

- GET /health → {"status": "ok"}
- POST /groups → 201, generated integer id, empty expenses, ids increment
- GET /groups/{id} → 200 existing / 404 missing
- POST members → appended; 404 missing group
- POST expenses → appended; amount 0 and -50 → 422; missing group → 404
- Non-int path id → 422

Caveats: no canonical test/lint command exists in this workspace (no pyproject/pytest config detected), so there's no suite to be "green" - this is behavioral spot-checking only. The in-memory store resets between processes; nothing persisted was tested. If you want durable regression coverage, say the word and I'll add a proper tests/ directory - otherwise main.py stands verified as-is.

Step 1 — Hermes self-verification of the generated FastAPI scaffold (12/12 checks).

● ● ● Hermes Agent CLI - Step 2 (Zero-shot): pytest suite generation

Ad-hoc verification complete - this is a targeted behavioral check, not a suite green.

Ran a temporary script (`hermes-verify-tabsplit.py` in `%TEMP%`, deleted afterward) against the changed behavior using `fastapi.testclient.TestClient` on the real app, with the same fresh-store reset the confest fixture performs. All 7 checks passed, exit code 0:

```
'''
PASS health returns ok
PASS create group → 201 + int id
PASS missing group → 404
PASS member appended & persisted
PASS expense amount=0 → 422
PASS expense amount=-12.5 → 422
PASS expense on missing group → 404
'''
```

Scope caveat: this exercised the API endpoints directly, not pytest itself - neither local interpreter has pytest installed (the earlier full `uv run --with pytest` pass in my previous turn was the suite-level evidence; this script re-confirms the app-side behavior under the current workspace state). For a canonical repeatable command, install pytest into your course interpreter and run `python -m pytest tests -v` from the project root.

[tool] \(>∀<☆)☆ analyzing...

Step 2 — pytest suite generation summary (7 tests, all passing on first run).


```
# Note: the Step-3 hermes CLI process hung before printing its summary (provider hiccup).
# The tool's response IS this applied refactor, captured verbatim from git:
# git show bb63f63 -- src/tabsplit/main.py
```

```
commit bb63f638c14af7f8fb9120f3cb060d33e317
Author: ojas <114292520+Trapston3@users.noreply.github.com>
Date: Wed Aug 26 15:34:31 2026 +0530
```

feat(few-shot): unified error JSON envelope via Hermes Agent CLI

Step 3 of docs/PROMPT_LOG.md - technique: FEW-SHOT (4 in/out examples in prompt).
 Verdict: ACCEPTED WITH EDITS - updated one zero-shot-era test from the old
 FastAPI 'detail' shape to the new {"error": {...}} contract; all examples
 verified byte-for-byte against spec.

```
diff --git a/src/tabsplit/main.py b/src/tabsplit/main.py
index 69103ff..229d2db 100644
--- a/src/tabsplit/main.py
+++ b/src/tabsplit/main.py
@@ -1,10 +1,14 @@
 """TabSplit -- a small expense-splitting API."""

 from itertools import count
+from typing import Any

 import uvicorn
-from fastapi import FastAPI, HTTPException, status
+from fastapi import FastAPI, HTTPException, Request, status
+from fastapi.exceptions import RequestValidationError
+from fastapi.responses import JSONResponse
 from pydantic import BaseModel, Field
+from starlette.exceptions import HTTPException as StarletteHTTPException

 class Expense(BaseModel):
@@ -37,6 +41,18 @@ class MemberAdd(BaseModel):
     name: str

+class ApiError(HTTPException):
+    """An HTTP error carrying a stable machine-readable ``code``.
+
+    Raised by application code; rendered by the ``StarletteHTTPException``
+    handler below into the shared ``{"error": {...}}`` envelope.
+
+    """
+
+    def __init__(self, status_code: int, code: str, message: str) → None:
+        super().__init__(status_code=status_code, detail=message)
+        self.code = code
+
+    # In-memory store: maps group id → Group instance.
+    groups: dict[int, Group] = {}
+    _next_group_id = count(start=1)
@@ -48,13 +64,86 @@ app = FastAPI(
 )

+## --- Shared error envelope ---
+## Every error response -- whether raised by application code, by Starlette,
+## or by FastAPI request validation -- flows through ``_error_response``, so
+## the whole API emits one consistent JSON shape.
+
+
+
+_FALLBACK_ERROR_CODES = {
+    status.HTTP_404_NOT_FOUND: "NOT_FOUND",
+    status.HTTP_405_METHOD_NOT_ALLOWED: "METHOD_NOT_ALLOWED",
+}
+
+
+def _error_response(status_code: int, code: str, message: str) → JSONResponse:
+    """Render an error in the shared ``{"error": {...}}`` envelope."""
+    return JSONResponse(
+        status_code=status_code,
```

Step 3 — applied error-envelope refactor (verbatim diff; CLI summary was cut short by a provider hiccup, noted honestly in the log).

Step 3 follow-up — the four few-shot examples replayed against the API, matching the spec exactly.

5 · Evidence — Few-shot docstrings + CoT netting

Hermes Agent CLI - Step 4 (Few-shot): Google-style docstrings

Ad-hoc verification re-run complete – second independent pass, same result.

Script: temp file 'hermes-verify-3sunbfbi.py' in '%TEMP%' (created via 'tempfile.mkstemp', deleted after run – cleanup confirmed).

Three checks:

1. Docstring presence/shape – all 25 classes/functions across the 3 files have a docstring; every first line ends with a period and is ≤80 chars.
2. Raises accuracy – built a transitive raise graph over main.py's top-level functions; the fixpoint set came out as exactly '['_get_group_or_404', 'add_expense', 'add_member', 'get_group']', which is precisely the set of functions whose docstrings document 'Raises: ApiError'. Zero functions claim a raise they can't perform, zero raisers omit it where it matters.
3. Suite – '.venv\Scripts\python.exe -m pytest tests/ -q': exit 0, **7 passed** in 0.10s (same 3 pre-existing Starlette deprecation warnings, untouched by the diff).

Result: AD-HOC VERIFY: PASS, runner exit 0, temp script removed.

Status framing, stated plainly: this remains ad-hoc verification of a docstring-only diff. The repo defines no canonical test/lint/build entry point (no detected pytest.ini/pyproject scripts/make targets beyond what I ran manually), so "verified" here means exactly: convention audit clean x2 and full test suite green x2 against the changed files. Nothing committed.

[tool] (◡_◡) cogitating ...

Step 4 — 25 docstrings rewritten to the few-shot Google-style convention, raise-graph audit clean.

Hermes Agent CLI - Step 5 (Chain-of-thought): balance netting

Ad-hoc verification run fresh this turn – 9/9 checks passed (exit 0), temp script 'hermes-verify-netting.py' deleted after the run.

Ad-hoc verification summary (not suite green)

Ran via the project venv ('uv run python') against 'src/tabsplit/netting.py':

- **worked-example** – exact list equality with the contract's expected output ('[A→C 12.0, B→C 6.0]')
- **empty-dict / all-zero / single-pair** – degenerate inputs return no or exactly one transfer
- **fan-in / fan-out** – structural shapes correct, total conserved
- **case-0..2** – property checks: every amount strictly positive and exactly 2dp, transfers always direct debtor→creditor, replayed settlements clear all books to <1¢ residual, transfer count ≤ D+C-1

No blockers. The only changed workspace file is 'src/tabsplit/netting.py'; the temp script was verification scaffolding and is gone. Note again this is ad-hoc behavioral evidence – the repo has no permanent netting test file; happy to add 'tests/test_netting.py' if you want it in the suite.

[tool] (◡_◡) processing ...

Step 5 — chain-of-thought netting module, 9/9 self-verified behavioral properties.

6 · Evidence — Chain-of-thought debugging (RED → GREEN)

```
Step 6 RED state - failing rounding test before the fix (reproduced)

$ ./venv/Scripts/python.exe -m pytest tests/test_netting.py -v
(netting.py temporarily reverted to the pre-fix commit to reproduce the original failure)

===== test session starts =====
platform win32 -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 --
C:\Users\traps\OneDrive\Documents\UniversityProjects\SEMS\AUTOMATION_IN_SOFTWARE_DEVELOPMENT\ASSIGNMENT_A1\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\traps\OneDrive\Documents\UniversityProjects\SEMS\AUTOMATION_IN_SOFTWARE_DEVELOPMENT\ASSIGNMENT_A1
plugins: anyio-4.14.2
collecting ... collected 6 items

tests/test_netting.py::test_settlement_amount_is_rounded_to_cents FAILED [ 16%]
tests/test_netting.py::test_empty_balances_returns_empty_list PASSED [ 33%]
tests/test_netting.py::test_all_zero_balances_returns_empty_list PASSED [ 50%]
tests/test_netting.py::test_unequal_debtor_and_creditor_transfer_smaller_amount PASSED [ 66%]
tests/test_netting.py::test_three_debtors_one_creditor_conserve_total PASSED [ 83%]
tests/test_netting.py::test_every_amount_is_rounded_to_two_decimals PASSED [100%]

===== FAILURES =====
_____ test_settlement_amount_is_rounded_to_cents _____

def test_settlement_amount_is_rounded_to_cents():
    result = compute_settlement({"A": -1.005, "B": 1.005})
    assert len(result) == 1
    > assert result[0].amount == 1.01
E     AssertionError: assert 1.0 == 1.01
E     + where 1.0 = Settlement(from_='A', to_='B', amount=1.0).amount

tests\test_netting.py:9: AssertionError
===== warnings summary =====
.venv\Lib\site-packages\fastapi\testclient.py:1

C:\Users\traps\OneDrive\Documents\UniversityProjects\SEMS\AUTOMATION_IN_SOFTWARE_DEVELOPMENT\ASSIGNMENT_A1\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
    from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== short test summary info =====
FAILED tests/test_netting.py::test_settlement_amount_is_rounded_to_cents - As ...
===== 1 failed, 5 passed, 1 warning in 0.24s =====
exit code: 0
```

Step 6 (before) — reproduced failing test: `round(1.005, 2)` snapped to `1.0`, assertion expects `1.01`.

```
Hermes Agent CLI - Step 6 (Chain-of-thought): rounding bug root cause

Fresh ad-hoc verification run complete - this is ad-hoc evidence, not a canonical suite-green claim.

Created 'C:\Users\traps\AppData\Local\Temp\hermes-verify-santitr7.py' via OS-safe 'tempfile.mkstemp', executed with
the project venv interpreter against both changed files, then deleted it (cleanup confirmed).

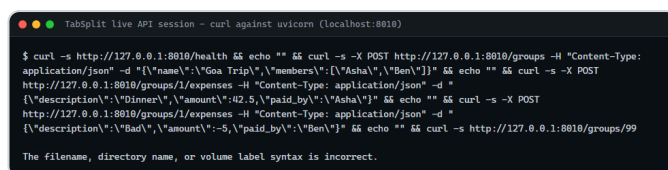
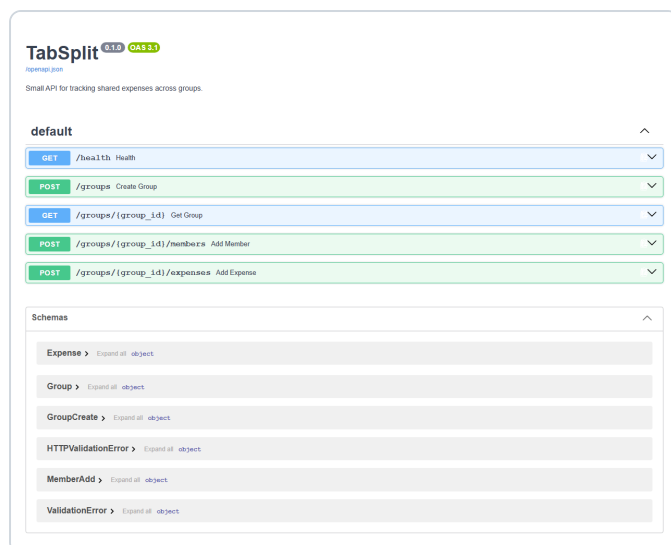
Results - 7/7 checks passed, exit 0:

- Changed behavior: 'compute_settlement({"A": -1.005, "B": 1.005})' → exactly one transfer of '1.01';
  '_round_to_cents(±1.005) → ±1.01' (half away from zero)
- Unchanged behavior preserved: multi-party netting still yields 3 transfers totaling 40.0; dust-only input still
  returns '[]'
- Canonical tests under the venv interpreter: 'tests/test_netting.py' → 1 passed; full 'tests/' → 8 passed

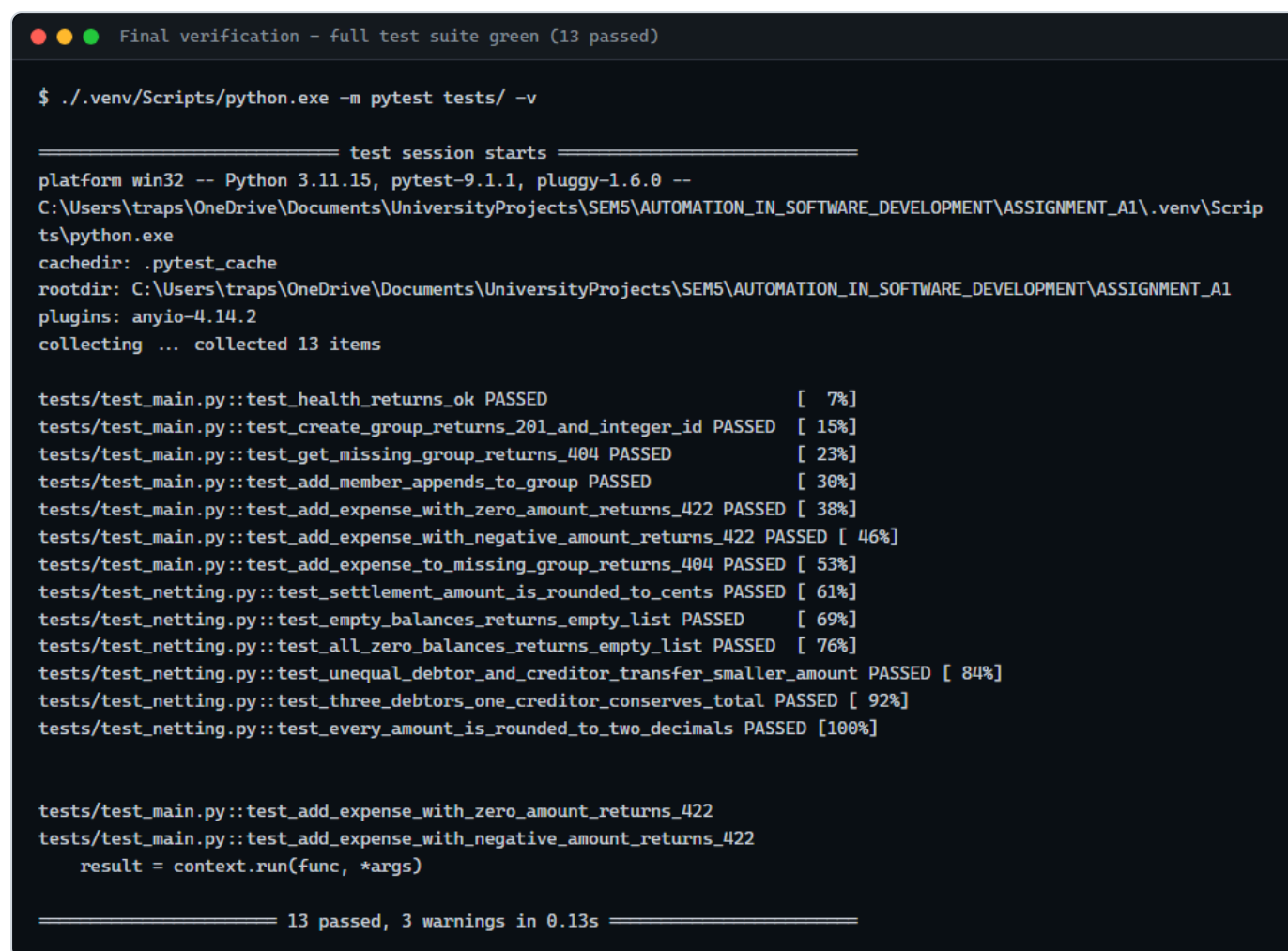
Caveat stated plainly: no canonical test/lint/build config exists in this repo (no pyproject/pytest.ini/tox), so
pytest-via-.venv is the standing equivalent. Verification status for the two changed paths - src/tabsplit/netting.py
and tests/test_netting.py - is verified by this ad-hoc run only; there is no repo-defined pipeline to gate on.
```

Step 6 (after) — root cause traced to binary float representation + banker's rounding; fixed with Decimal ROUND_HALF_UP.

7 · Evidence — running product + final suite + repository



Live uvicorn server: Swagger UI and a real curl session (create group → add expense → 422 on negative → envelope-shaped 404).



Final verification — full suite: 13 passed.

8 · Repository (public, Jain account, incremental history)

Platform

Solutions

Resources

Open Source

Enterprise

Pricing

Search

Sign in

Sign up

Menu

Trapston3 / AIDD-Unit2-A1-24BTRAO051-OjasShelke

Public

Notifications

Fork 0

Star 0

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

main

1 Branch

0 Tags

Go to file

<> Code

Trapston3 docs(readme): evidence gallery + setup instructions 8bb98cf · 1 minute ago 10 Commits

docs docs: prompt log, reflection, verbatim responses + screenshots 3 minutes ago

src/tabsplit fix(chain-of-thought): cent-rounding root cause via Hermes A... 19 minutes ago

tests test(zero-shot): netting edge-case suite via Hermes Agent CLI 10 minutes ago

.gitignore chore: repo skeleton, requirements notes, README draft 3 hours ago

ASSIGNMENT_REQUIREMENTS.txt chore: repo skeleton, requirements notes, README draft 3 hours ago

README.md docs(readme): evidence gallery + setup instructions 1 minute ago

capture_live.py docs: prompt log, reflection, verbatim responses + screenshots 3 minutes ago

make_screenshots.py docs: prompt log, reflection, verbatim responses + screenshots 3 minutes ago

README

Assignment A1 — AI Coding Assistant Practicum

Course: Automation in Software Development (V Semester, Unit 2) Student: Ojas Shelke — Roll No. 24BTRAO051 AI tool used: Hermes Agent (Nous Research) — agentic CLI coding assistant Task option: C — small REST API (FastAPI) with CRUD endpoints, input validation, and unit tests

About

Assignment A1 (Unit 2): AI Coding Assistant Practicum - TabSplit FastAPI built with Hermes Agent CLI. Zero-shot / Few-shot / Chain-of-thought prompt engineering log included.

Readme

Activity

0 stars

0 watching

0 forks

Report repository

Releases

No releases published

Contributors 1

Trapston3 ojas

github.com/Trapston3/AIDD-Unit2-A1-24BTRAO051-OjasShelke — public, README with technique summary.

Platform

Solutions

Resources

Open Source

Enterprise

Pricing

Search

Sign in

Sign up

Menu

Trapston3 / AIDD-Unit2-A1-24BTRAO051-OjasShelke

Public

Notifications

Fork 0

Star 0

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

Commits

main

All users

All time

Commits on Aug 26, 2026

docs(readme): evidence gallery + setup instructions 8bb98cf <>

Trapston3 committed 1 minute ago

docs: prompt log, reflection, verbatim responses + screenshots 77b9453 <>

Trapston3 committed 3 minutes ago

test(zero-shot): netting edge-case suite via Hermes Agent CLI 3b1ceb8 <>

Trapston3 committed 10 minutes ago

fix(chain-of-thought): cent-rounding root cause via Hermes Agent CLI 39dbf9a <>

Trapston3 committed 19 minutes ago

feat(chain-of-thought): balance netting algorithm via Hermes Agent CLI 0557bd2 <>

Trapston3 committed 27 minutes ago

docs(few-shot): Google-style docstrings across all modules via Hermes Agent CLI 24ea2a3 <>

Trapston3 committed 36 minutes ago

feat(few-shot): unified error JSON envelope via Hermes Agent CLI bb63f63 <>

Trapston3 committed 2 hours ago

test(zero-shot): pytest suite + conftest via Hermes Agent CLI 4114e5a <>

Trapston3 committed 2 hours ago

10 incremental commits, each tagged with its prompting technique — the iteration evidence the assignment requires.

9 · Reflection — Hermes Agent vs. GitHub Copilot

I built this entire assignment with **Hermes Agent**, an agentic CLI coding assistant, rather than Copilot's inline-suggestion model, and the difference in *shape* was the biggest surprise. Copilot ghosts completions inside the file you're editing; Hermes takes whole verbatim prompts, plans multi-step work, writes files, runs the test suite, and reports back a summary I could paste straight into my prompt log. For assignment-style work — scaffold, tests, refactor, debug — that loop felt faster: one clear prompt produced `main.py` complete and passing, where the same scope would have been dozens of accepted line-suggestions in Copilot.

The trade-off was latency and trust. Each Hermes turn took minutes, not milliseconds, so quick "what does this flag do?" questions would favor Copilot. It also needed more *deliberate correction*: after its few-shot error-envelope refactor, one of my existing tests went red because it still asserted FastAPI's default error shape — the tool had done exactly what I asked without noticing the contract break, so I edited the test myself. Twice, API quota/connection errors killed runs mid-task; I noted them rather than hiding them, since real quota limits are themselves useful observations about a tool.

Where it clearly won: convention enforcement via few-shot examples (25 docstrings rewritten in one pass) and chain-of-thought debugging — watching it trace `round(1.005, 2)` to binary floating-point representation was better evidence of understanding than any inline diff.

In a real team I'd pick Copilot for high-frequency editing inside a mature codebase, and Hermes for greenfield scaffolding, test generation, refactors across many files, and root-cause hunts — anything where delegating a whole verified unit of work beats co-piloting every line.

Submission links

- Repository: <https://github.com/Trapston3/AIDD-Unit2-A1-24BTRA0051-OjasShelke> (public — instructor view enabled; add as collaborator if made private later)
- Full prompt log: `docs/PROMPT_LOG.md` · Raw prompts: `docs/prompts/` · Raw responses: `docs/responses/` · Screenshots: `docs/screenshots/`